

TP d'Algorithmique

N°8

Table des matières :

Exercice 1 :	2
I : Menu :	2
1 : Code :	2
2 : Jeux d'essais :	4
II : Tri Bulle :	5
1 : Code :	5
2 : Jeux d'essais :	5
III : Tri insertion :	7
1 : Code :	7
2 : Jeux d'essais :	7
IV : Recherche dichotomique :	9
1 : Code :	9
2 : Jeux d'essais :	9

Exercice 1 :

Ce programme a pour objectif d'être un menu permettant la saisie et l'affichage d'un tableau de N cases rempli par des entiers aléatoires. Il peut aussi être trié de 2 manières différentes, le tri à bulles et le tri par insertion. Et enfin il possède une fonction de recherche dichotomique d'un élément choisi par l'utilisateur.

Le menu bloque aussi les choix de tri si le tableau n'a pas été créé. Il bloque aussi la recherche dichotomique si le tableau n'a pas été trié au préalable. Grâce aux fonctions booléennes : `tableau_creer` et `tableau_trier`.

I : Menu :

Le programme principal ne possède que le menu, les tris et les affichages sont via des fonctions importées. Il fonctionne via un while qui changera à False si le choix est 6 donc "Quitter" ce qui permettra d'arrêter le programme. Le reste du temps les autres choix feront leur rôle respectif.

1 : Code :

```
if __name__ == "__main__":
    choix : int
    continuer : bool
    tableau : list[int]
    tableau_creer : bool
    tableau_trier : bool

    tableau = []
    tableau_creer = False
    tableau_trier = False
    continuer = True

    while continuer == True :
        print("Bienvenue dans le programme de tri de tableau")
        choix = int(input("1. Créer un nouveau tableau\n2. Afficher le tableau de manière lisible\n3. Trier le tableau (Tri à bulle) + afficher temps\n4. Trier le tableau (Tri inserti

        while choix < 1 or choix > 6:
            print("Veuillez choisir un nombre entre 1 et 6")
            choix = int(input("1. Créer un nouveau tableau\n2. Afficher le tableau de manière lisible\n3. Trier le tableau (Tri à bulle) + afficher temps\n4. Trier le tableau (Tri ins

        if choix == 1:
            n = int(input("Veuillez entrer la taille du tableau à créer: "))
            tableau = creer_tableau(n)
            tableau_creer = True
            print("-----")
            print("\033[0;32mTableau créé \033[0m\n")

        if choix == 2 and tableau_creer == True :
            afficher_tableau(tableau)
            print("\n")

    if choix == 3 and tableau_creer == True:
        debut = time()
        tri_bulles(tableau)
        fin = time()
        tableau_trier = True
        print("-----")
        print("\033[0;32mTableau trié avec tri bulle avec succès en",fin - debut,"secondes \033[0m")
        print("\n")

    if choix == 4 and tableau_creer == True:
        debut = time()
        tri_insertion(tableau)
        fin = time()
        tableau_trier = True
        print("-----")
        print("\033[0;32mTableau trié avec tri insertion avec succès en",fin - debut,"secondes \033[0m")
        print("\n")

    if choix == 5 and tableau_creer == True and tableau_trier == True :
        recherche_dicho(tableau, int(input("Veuillez entrer le nombre à rechercher: ")))
        print("-----")
        print("\033[0;32mRecherche d'un élément dans un tableau trié \033[0m")
        print("\n")

    if choix != 1 and tableau_creer == False :
        print("-----")
        print("\033[0;31mVeuillez créer un tableau avant de continuer \033[0m")

    if choix == 6:
        print("Au revoir")
        continuer = False
```

2 : Jeux d'essais :

```
1. Créer un nouveau tableau
2. Afficher le tableau de manière lisible
3. Trier le tableau (Tri à bulle) + afficher temps
4. Trier le tableau (Tri insertion) + afficher temps
5. Rechercher un élément dans un tableau trié
6. Quitter
--> 1
Veuillez entrer la taille du tableau à créer: 20
-----
Tableau créé
```

```
Bienvenue dans le programme de tri de tableau
1. Créer un nouveau tableau
2. Afficher le tableau de manière lisible
3. Trier le tableau (Tri à bulle) + afficher temps
4. Trier le tableau (Tri insertion) + afficher temps
5. Rechercher un élément dans un tableau trié
6. Quitter
--> 2
Valeur 0 = 92

Valeur 1 = 18

Valeur 2 = 34

Valeur 3 = 23

Valeur 4 = 30

Valeur 5 = 78

Valeur 6 = 37

Valeur 7 = 96

Valeur 8 = 25

Valeur 9 = 25

Valeur 10 = 94

Valeur 11 = 9

Valeur 12 = 10

Valeur 13 = 36

Valeur 14 = 69

Valeur 15 = 51

Valeur 16 = 19

Valeur 17 = 45

Valeur 18 = 87

Valeur 19 = 94
```

L'affichage "valeur" correspond à l'affichage de la valeur à l'indice i de la liste après tri.

```
Bienvenue dans le programme de tri de tableau
1. Créer un nouveau tableau
2. Afficher le tableau de manière lisible
3. Trier le tableau (Tri à bulle) + afficher temps
4. Trier le tableau (Tri insertion) + afficher temps
5. Rechercher un élément dans un tableau trié
6. Quitter
--> 6
Au revoir
```

II : Tri Bulle :

1 : Code :

```
def tri_bulles(liste : list[int]) -> list[int] :
    """
    Trie un tableau par tri à bulles
    Args:
        liste (list[int]): Tableau à trier
    Returns:
        list[int]: Tableau trié
    """

    p : int
    echange : bool
    #echange indique à la fonction si elle doit continuer son tri
    tmp : int
    i :int
    p = len(liste)-1
    echange = True
    while echange and p > 0 :
        echange = False
        for i in range(0, p) :
            if liste[i] > liste[i+1] :
                tmp = liste[i]
                liste[i] = liste[i+1]
                liste[i+1] = tmp
                echange = True
        p = p-1
    return liste
```

2 : Jeux d'essais :

```
Bienvenue dans le programme de tri de tableau
1. Créer un nouveau tableau
2. Afficher le tableau de manière lisible
3. Trier le tableau (Tri à bulle) + afficher temps
4. Trier le tableau (Tri insertion) + afficher temps
5. Rechercher un élément dans un tableau trié
6. Quitter
--> 3
-----
Tableau trié avec tri bulle avec succès en 2.5033950805664062e-05 secondes
```

```
Valeur 0 = 9
Valeur 1 = 10
Valeur 2 = 18
Valeur 3 = 19
Valeur 4 = 23
Valeur 5 = 25
Valeur 6 = 25
Valeur 7 = 30
Valeur 8 = 34
Valeur 9 = 36
Valeur 10 = 37
Valeur 11 = 45
Valeur 12 = 51
Valeur 13 = 69
Valeur 14 = 78
Valeur 15 = 87
Valeur 16 = 92
Valeur 17 = 94
Valeur 18 = 94
Valeur 19 = 96
```

L’affichage “valeur” correspond à l’affichage de la valeur à l’indice i de la liste après le tri à bulle.

III : Tri insertion :

1 : Code :

```
def tri_insertion(liste : list[int]) -> list[int] :  
    """  
    Trie un tableau par insertion  
    Args:  
        liste (list[int]): Tableau à trier  
    Returns:  
        list[int]: Tableau trié  
    """  
    val :int  
    #val sert à stocker des valeurs pour permettre l'échange des valeurs de donnée du tableau  
    i : int  
    j : int  
    n : int  
    n = len(liste)  
    for i in range(1, n) :  
        val = liste[i]  
        j = i  
        while j > 0 and liste[j-1] > val :  
            liste[j] = liste[j-1]  
            j = j-1  
        liste[j] = val  
    return liste
```

2 : Jeux d'essais :

```
Bienvenue dans le programme de tri de tableau  
1. Créer un nouveau tableau  
2. Afficher le tableau de manière lisible  
3. Trier le tableau (Tri à bulle) + afficher temps  
4. Trier le tableau (Tri insertion) + afficher temps  
5. Rechercher un élément dans un tableau trié  
6. Quitter  
--> 4  
-----  
Tableau trié avec tri insertion avec succès en 4.124641418457031e-05 secondes
```

```
Valeur 0 = 2
Valeur 1 = 20
Valeur 2 = 28
Valeur 3 = 29
Valeur 4 = 31
Valeur 5 = 32
Valeur 6 = 33
Valeur 7 = 35
Valeur 8 = 39
Valeur 9 = 41
Valeur 10 = 44
Valeur 11 = 56
Valeur 12 = 69
Valeur 13 = 79
Valeur 14 = 81
Valeur 15 = 84
Valeur 16 = 91
Valeur 17 = 91
Valeur 18 = 95
Valeur 19 = 98
```

L'affichage "valeur" correspond à l'affichage de la valeur à l'indice i de la liste après tri.

IV : Recherche dichotomique :

La recherche dichotomique consiste à couper le tableau en 2 parties est faire une recherche pour savoir de quel côté du tableau est la valeur recherchée, puis de recouper le tableau en 2 mais cette fois à partir de la borne du côté de la valeur. On recommence l'opération jusqu'à, soit trouver la valeur ,soit avoir fini la recherche.

1 : Code :

```
#Cette fonction permet de savoir si le tableau rentré dans la recherche est trié.
def est_trié(liste : list[int]) -> bool :
    """
    Vérifie si un tableau est trié
    Args:
        liste (list[int]): Tableau à vérifier
    Returns:
        bool: True si le tableau est trié, False sinon
    """

    i : int
    tri : bool
    tri = True
    for i in range(len(liste)-1) :
        if liste[i] > liste[i+1] :
            tri = False
    return tri
```

```
def recherche_dicho(liste : list[int], x : int) -> int :
    """
    Recherche un élément dans un tableau trié
    Args:
        liste (list[int]): Tableau trié
        x (int): Element à rechercher
    Returns:
        int: Indice de l'élément recherché, -1 si non trouvé
    """
    #a correspond à la partie gauche du tableau, b la partie droite et m le milieu de a et b
    #p est une variable annexe qui sert juste pour le retour de la donnée s'incrémentant de la valeur de la position de la donnée souhaiter
    a : int
    b : int
    m : int
    p : int
    p = -1
    a = 0
    b = len(liste) - 1
    m = (a+b)//2
    if est_trié(liste) :
        while a < b :
            m = (a+b)//2
            if x <= liste[m] :
                b = m
            else:
                a = m+1
            if liste[a] == x :
                p = a
            else :
                p = -1
            print("Le numéro est à l'indice : ", p, "\nOu sinon à la position : ", p+1)
            return p
    else :
        print("Le tableau n'est pas trié")
        return p
```

2 : Jeux d'essais :

```
Veillez entrer le nombre à rechercher: 7
Le numéro est à l'indice : -1
Ou sinon à la position : 0
-----
Recherche d'un élément dans un tableau trié
```

```
Veillez entrer le nombre à rechercher: 41
Le numéro est à l'indice : 9
Ou sinon à la position : 10
-----
Recherche d'un élément dans un tableau trié
```